Practical – 13

Practical Title : PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory :

1. Do PHP endpoints return valid JSON responses?

Yes, the PHP API returns responses in JSON format. It uses json_encode() to convert data into JSON. The response contains a success status, message and data. It returns event records when a GET request is received. It also returns JSON error messages when the database connection fails or invalid input is provided.

2. Are GET and POST requests handled properly?

Yes, GET and POST requests are handled using PHP. The GET request retrieves all events from the database. The POST request performs Create, Update and Delete operations according to the action received. Prepared statements are used for database operations. Unsupported request methods return an appropriate error response.

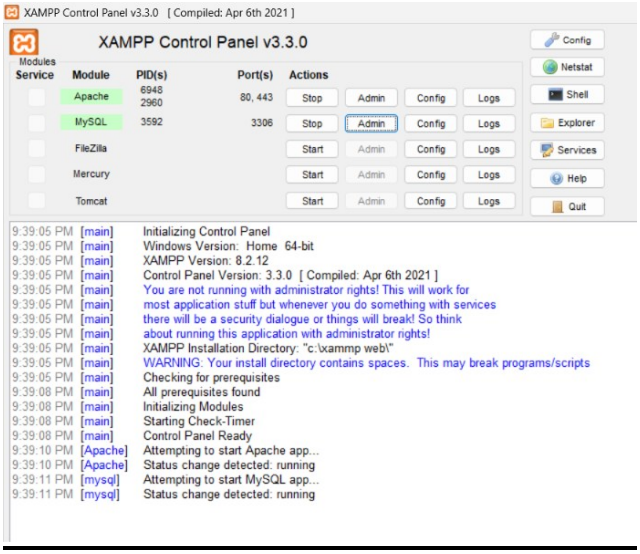
3. Are loading, success, and error states shown on the frontend?

Yes, the frontend displays a loading message while fetching event data. Success messages appear after successful operations. Error messages are displayed when API requests fail or the database is unavailable. The Fetch API handles the responses using try...catch. This helps users understand the status of each operation.

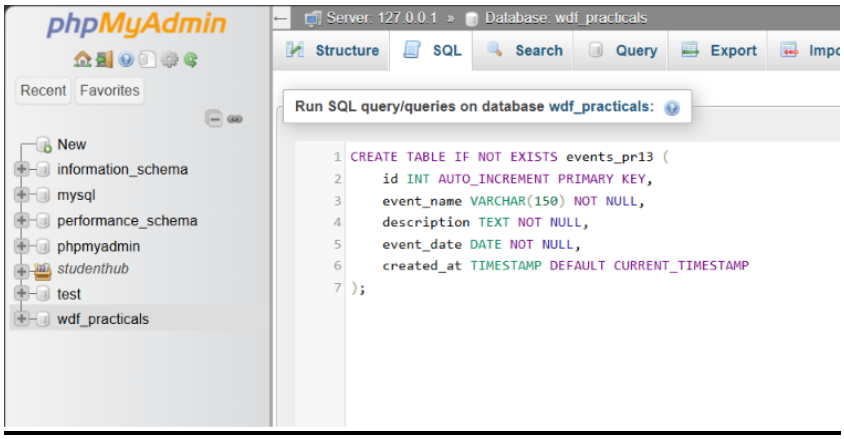
4. Is input validation performed before database operations?

Yes, input validation is performed in the PHP API before database operations. It checks that the event name, description and date are not empty. The event name length and date format are also validated. The event ID is checked before updating or deleting records. Prepared statements help protect the database against SQL injection.

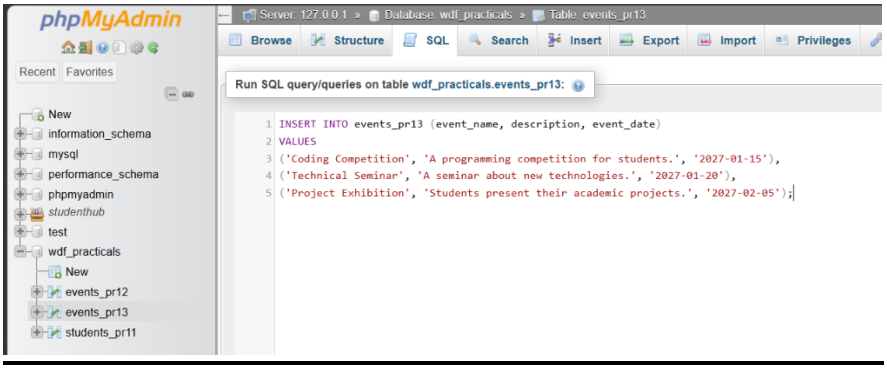
(1)Start XAMMP Local Host Server(APACHE and MYSQL Start):



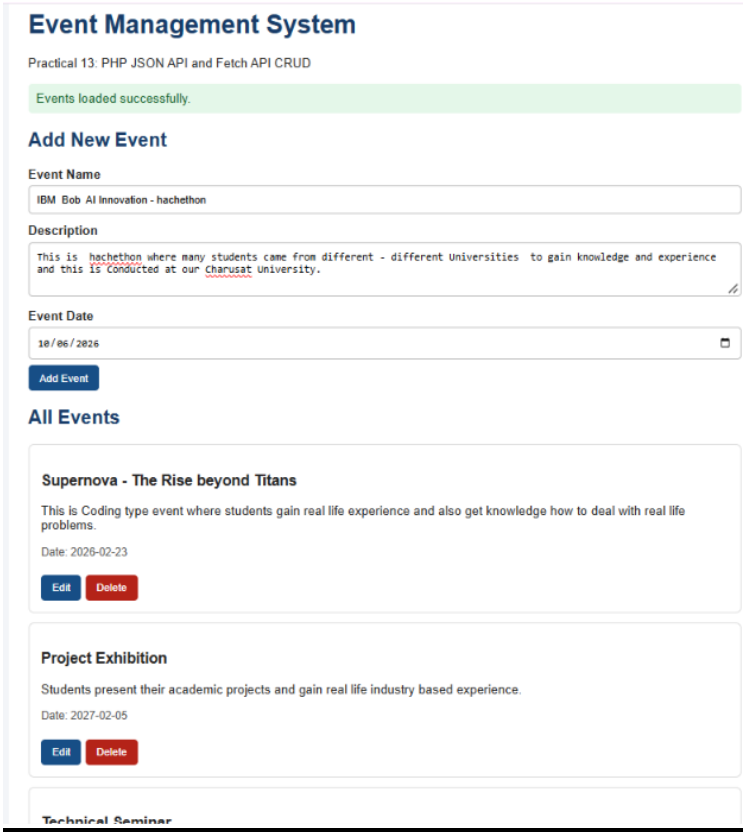
(2)For Event add, edit , delete write SQL query in database :



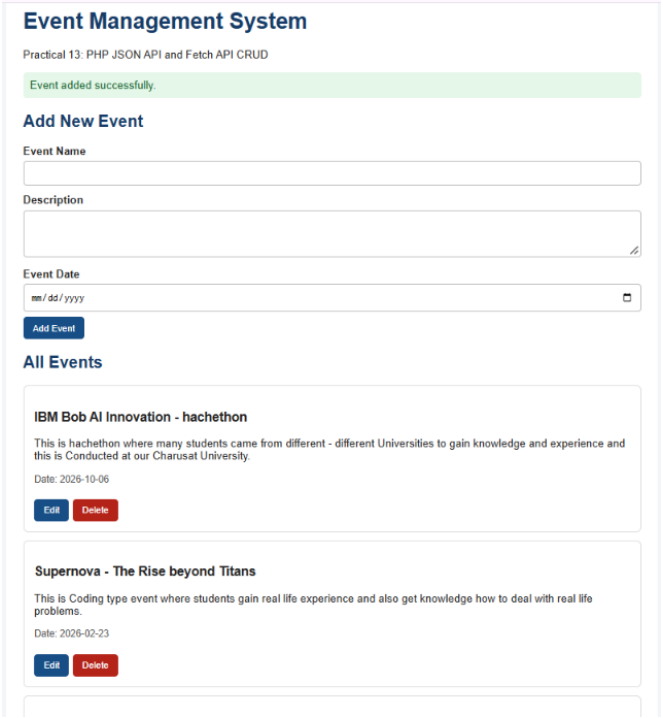
Add some events by inserting SQL query in Database:



(3)Add New Event :

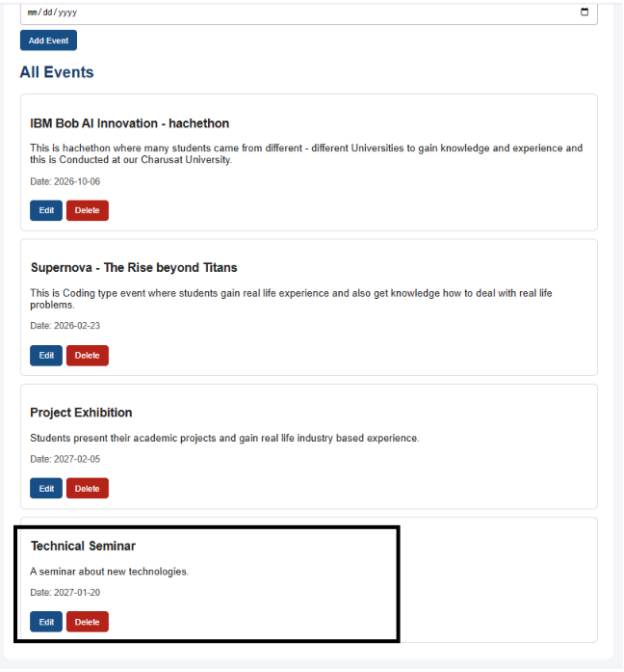


(4)Event Added Successfully :

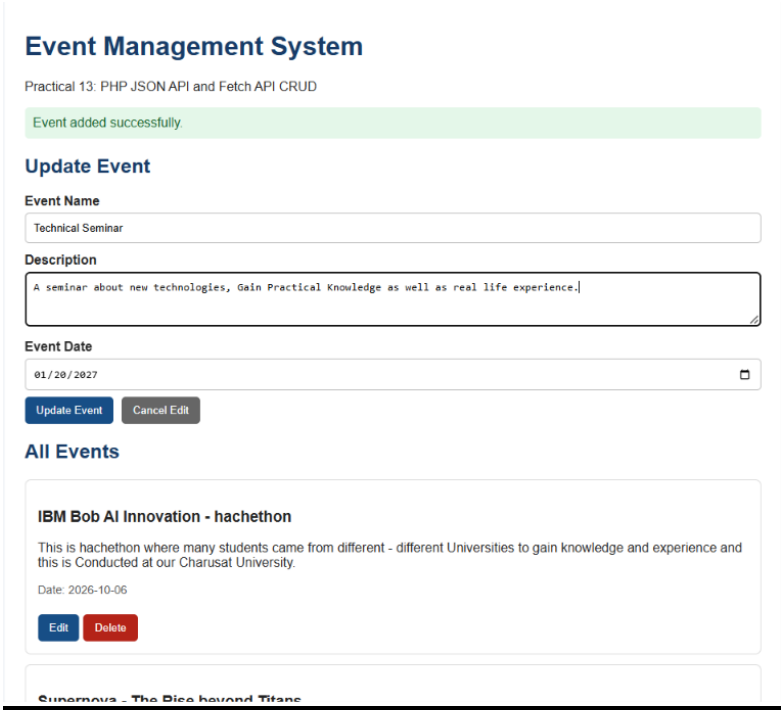


(5)Edit event :

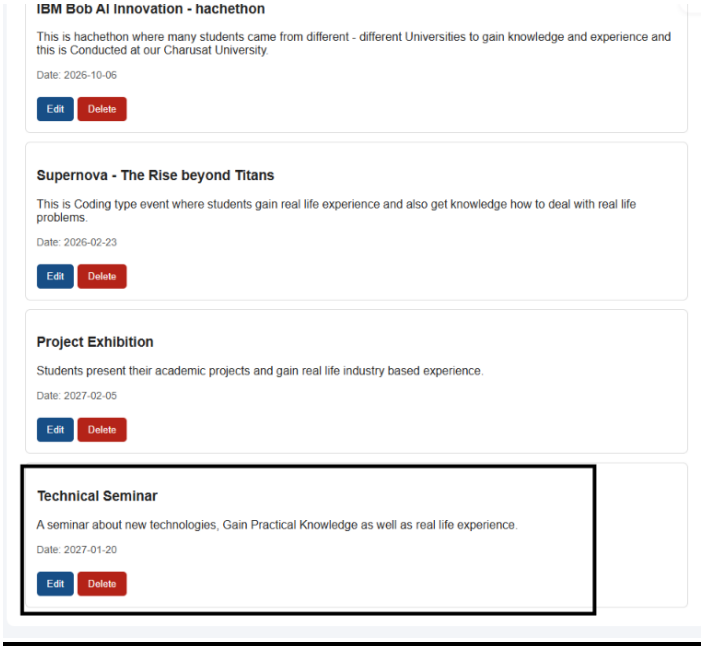
Before Edit event:



Edit:

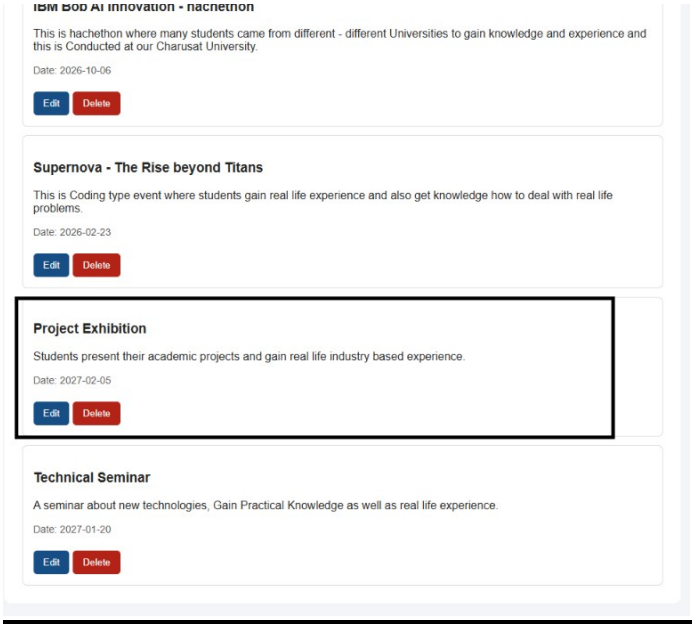


After Edit Details:

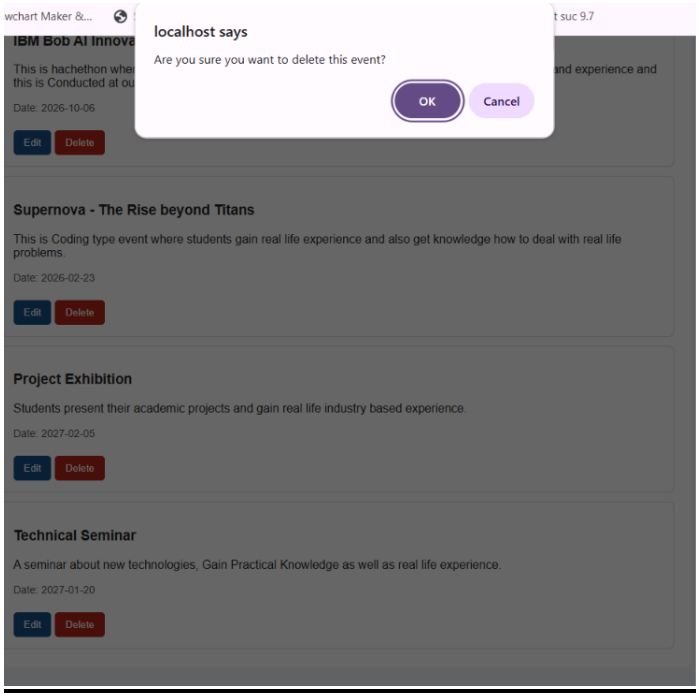


(6)Delete event:

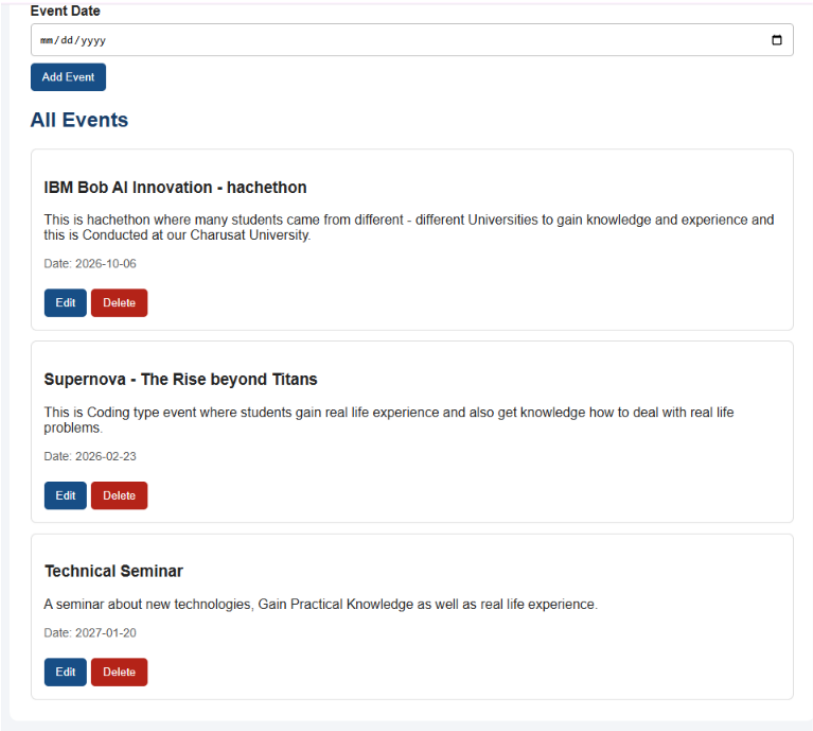
Before Delete:



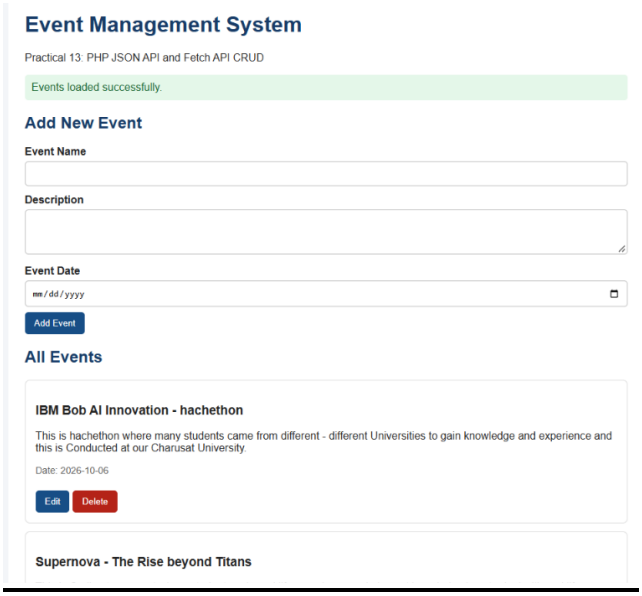
Delete:



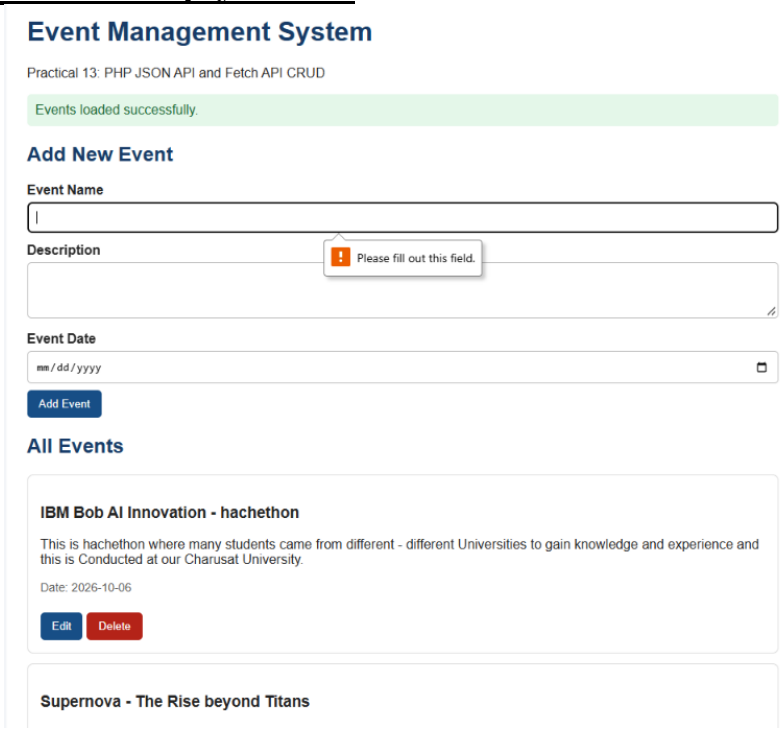
After Delete event:



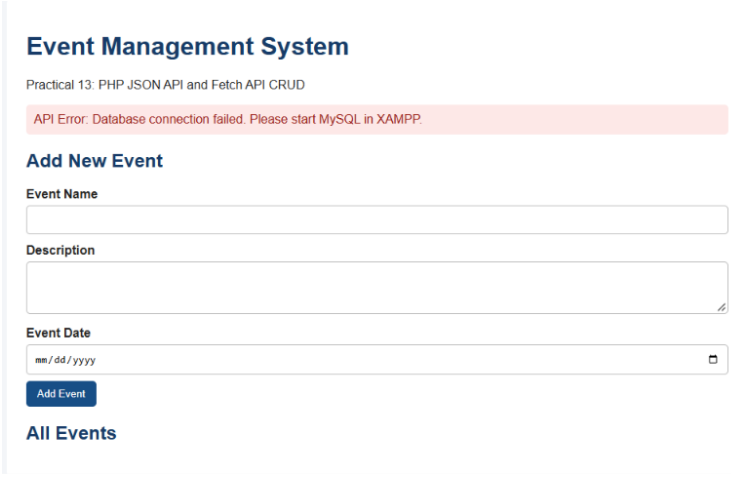
(7)Validation: Try to submit empty form :



We cannot submit empty form :



(8)Temporary stop MYSQL and check for event so it will give Error:



GitHub Repository Link:

https://github.com/MeetBhatti/WDF.git

Learning Outcomes:

After completing this practical, I learned how to create a PHP JSON API and connect it with a MySQL database using MySQLi. I learned how to send and receive data using the JavaScript Fetch API. I performed Create, Read, Update and Delete operations without reloading the complete webpage. I understood how JSON transfers information between PHP and JavaScript. I also learned to validate input, use prepared statements and display success or error messages. Finally, I learned how to inspect API requests and responses using the browser's Network tab.

Faculty Remarks: (Keep this blank for faculty)